[2025-1 데이터구조(백승환 교수님) 중간고사] 복기 - 25 문준호

1.
(1) 배열 포인터, 주소 관련 코드가 주어지고 각각의 출력 결과를 써라. (a~f),
a~c는 배열 포인터와 주소 관련 문제. d~f는 casting 관련 문제.

* 코드는 복기를 못했습니다. ㅠㅠ

```
<알아두면 좋은 것>

(a-c) 여기서 출력결과 뭐나오는지
int arr[] = {1, 5, 10, 20, 30};
int *p = arr[0];
cout << p;
p = arr;
cout << p;
cout << (p++);
cout << p;
cout << (++p);

(d-f)
int를 float으로 나누면 반환형 뭔지
[int형 변수 x]에 [int를 float으로 나눈 결과]를 대입하면 어떻게 되는지
```

(2) &arr[3]가 의미하는 바를 써라.

2.
(1) 코드 보고 해당 코드의 실행 시간을 n에 대한 함수로 나타낸 것을 T(n)이라고 하자. 이때 T(n)을 구하라. --> 정답 : 2n-1

(2) (1)에서 구한 T(n)이
(a) Big-Oh 조건을 만족하도록 할 수 있는가? 만족할 수 있는지 판별하고, 만족할 수 있다면 상수 c와 n_0를 결정하라. --> 정답 : 있다.
(b) Big-Omega 조건을 만족하도록 할 수 있는가? 만족할 수 있는지 판별하고, 만족할 수 있다면 상수 c와 n_0를 결정하라. --> 정답 : 없다.
(c) Big-Theta 조건을 만족하도록 할 수 있는가? 만족할 수 있는지 판별하고, 만족할 수 있다면 상수 c와 n_0를 결정하라. --> 정답 : 없다.

3. Pointer-Based List를 One-step ahead Method로 구현하고자 한다. 삽입, 삭제 함수에 대한 pseudo-code를 써라.

(1) Insertion

```
void InsertNode(int* position, int value){




}
```

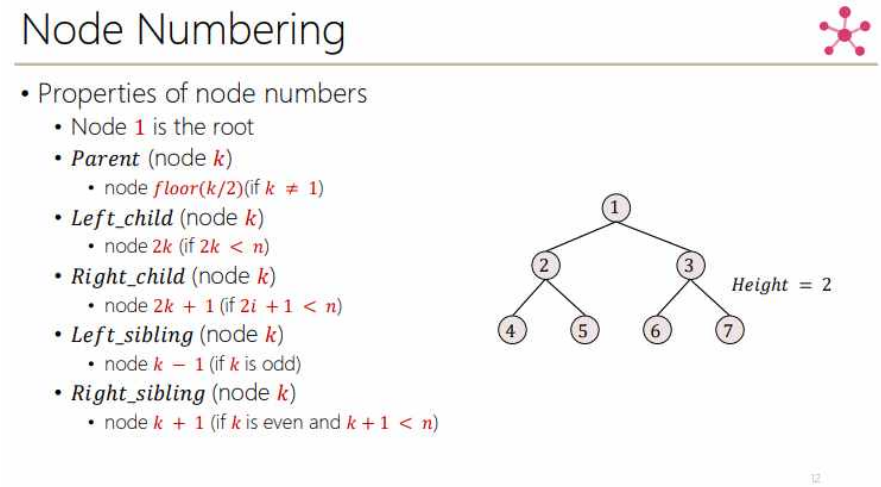
(2) Delete

```
void DeleteNode(int* position){




}
```

4.

(1) Tree에서 Parent, Left_child, Right_child, Left-sibling, Right_sibling을 구하는 함수를 구현하고자 한다. 각각의 함수에서 빈칸을 채워넣어라.

*참고:



이 슬라이드에서 빨간색 글자들이 함수 내부 빈칸으로 나옴.

(조건(if문)과, 무엇을 return하는지)

(2) Tree를 Array-based Method로 구현했을 때와 Pointer-Based Method로 구현했을 때 각각의 장단점을 1가지씩 써라.

--> 정답:

-Array-based Method

장점 : Cache Coherence의 관점에서 memory access time이 작다.

단점 : 사이즈가 급격하게 커졌을 때 동적으로 대응하기 어렵다. (더 큰 배열을 만들고 원래 있던 배열에서 값을 복사해주어야 하는데 이때 cost가 발생하므로)

-Pointer-Based Method

Array-based Method의 장단점을 바꿔서 쓰면 됨.

5. Min-Heap은 Structural Property와 Heap Order Property를 만족해야 한다.

(1) 각각의 Property가 어떤 걸 의미하는지 간략하게 설명하라.

(a) Structural Property : _____

(b) Heap Order Property : _____

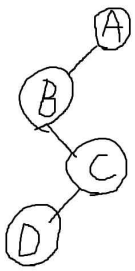
(2) Heapifying CBT를 하는 1)방법과 2)시간복잡도에 대해 언급하고, 시간복잡도의 경우 그 이유도 간략하게 설명하라.

--> 정답 :

1) Start with the rightmost array position k that has a child
(the last non-terminal node at index $k = floor(\frac{n}{2})$, n is # of nodes)
2) If the subtree rooted at k is not a min-heap
Rearrange the subtree (by pushing down k until it reaches a level where it is less than its children, or is a leaf node)
3) Repeat the step-2 at $k-1, k-2, \ldots, 1$ (from the high index of the array to the low index)

- Number (subtrees at level k) $\leq 2^k$
- Time (each subtree) = $O(h-k)$
- Time (all subtrees at level k) $\leq 2^k (h-k)$
- Total time = $O(\sum_{k=0}^{h-1} 2^k (h-k)) = \ldots = O(2^h) = O(n)$

6. 주어진 Binary Tree는 Knuth Transform에 의해 변환된 Binary Tree이다.



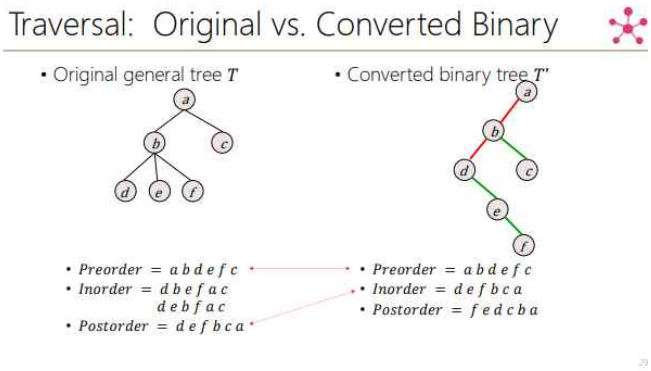
(1) 변환되기 전 Original Binary Tree를 그려라.

(2) (1)의 Original Tree를 Postorder한 결과와, 문제에 주어진 Converted Binary Tree를 Inorder한 결과를 각각 써라.

-Postorder : _____

-Inorder : _____

*참고 : 교수님 ppt에도 나와있듯이, 두 개의 답이 같습니다.



7. Quick Sort를 In-place하게 구현하고자 한다. 주어진 코드에서 빈칸을 채워 넣으세요.

인터넷에서 최대한 비슷한 코드를 찾아봤습니다.

```
Quicksort (A as array, low as int, high as int)
    if (low < high)
        pivot_location = Partition (A, low, high)
        Quicksort (A, low, pivot_location)
        Quicksort (A, pivot_location + 1, high)
```

```
Partition (A as array, low as int, high as int)
    pivot = A[low]
    leftwall = low

    for i = low + 1 to high
        if (A[i] < pivot) then
            swap (A[i], A[leftwall])
            leftwall = leftwall + 1

    swap (pivot, A[leftwall])

    return (leftwall)
```

빨간색 네모친 부분이 빈칸으로 나왔습니다.

8. BST

(몇 개의 원소가 삽입되어져 있는 BST가 그림으로 주어짐)

(1) 주어진 BST에서, 17과 30을 차례로 삽입한 뒤 결과를 각각 그리세요.

(a) 17 삽입 후 (b) 30 삽입 후

(2) n개의 원소가 있는 BST에서 삽입 연산을 수행할 때 시간 복잡도는 어떻게 되는가? Best Case와 Worst Case에 대해 각각 쓰고, 그 Case는 어떤 Case인지 설명하라.

(a) Best Case : _____

(b) Worst Case : _____

(3) 주어진 BST에서, 5와 12를 차례로 삭제한 뒤 결과를 각각 그리세요. (단, (1)에서 17과 30을 추가하기 이전, Original Tree 상태에서 삭제 연산을 수행한다.)

(a) 5 삭제 후 (b) 12 삭제 후

(4) BST에서 Delete 함수를 구현한 코드이다. 코드의 빈칸을 채워라.
최대한 비슷한 코드를 찾아봤습니다. 빨간색 네모가 빈칸으로 나왔습니다.

```
/* A recursive function to
  delete an existing key in BST
 */
Node deleteRec(Node root, int key)
{
    /* Base Case: If the tree is empty */
    if (root == null)
        return root;

    /* Otherwise, recur down the tree */
    if (key < root.key)
        root.left = deleteRec(root.left, key);
    else if (key > root.key)
        root.right = deleteRec(root.right, key);

    // if key is same as root's key, then This is the
    // node to be deleted
    else {
        // node with only one child or no child
        if (root.left == null)
            return root.right;
        else if (root.right == null)
            return root.left;

        // node with two children: Get the
        // inorder successor (smallest
        // in the right subtree)
        root.key = minValue(root.right);

        // Delete the inorder successor
        root.right = deleteRec(root.right, root.key);
    }
```

*참고사항
1. 169점 만점.

- Max score : 151
- Min score : 0
- Mean : 88.8
- Std : 36.4
- Median : 91

2. 손코딩 문제의 점수 배점이 높습니다. (문제당 20~30점 정도)
코드는 교수님 ppt에서 다루지 않으니, 배운 내용들의 c++ 코드를 인터넷에서 한번 찾아보고 가면 좋습니다.
ex) Sort 함수들, BST 함수, Heap에서 함수, LinkedList 삽입 삭제 연산 함수 등...